

1. Descrição da Implementação

Ambas as implementações resolvem o mesmo problema, mas utilizam modelos de concorrência fundamentalmente diferentes.

1.1. Versão preemptiva (threads)

Esta versão utiliza o módulo `threading` do Python para criar múltiplos threads reais do Sistema Operacional (SO), que são agendados de forma preemptiva (o SO pode interrompê-los a qualquer momento).

Para garantir a segurança do buffer compartilhado (`collections.deque`) são utilizadas primitivas de sincronização clássicas:

- **`threading.Lock (Mutex)`:** que garante a exclusão mútua. Ele protege a "seção crítica" (as funções `produz()` e `consome()`), assegurando que apenas um thread possa modificar o buffer por vez.
- **`threading.Semaphore (Semáforo empty)`:** Um semáforo contador que é inicializado com o tamanho do buffer (`tamanho_buffer`). Ele é usado pelos produtores para esperar (`.acquire()`) por um espaço vazio. Se o buffer estiver cheio, o semáforo bloqueia o produtor.
- **`threading.Semaphore (Semáforo full)`:** Um semáforo contador inicializado em 0. Ele é usado pelos consumidores para "esperar" (`.acquire()`) por um item. Se o buffer estiver vazio, o semáforo bloqueia o consumidor.
- **Parada:** A parada dos consumidores rodam em um `while True`, onde a `main` insere `None` no buffer para cada consumidor após os produtores terminarem.

1.2. Versão cooperativa (coroutines)

Esta versão utiliza o módulo `asyncio` e roda em um **único thread**. A concorrência é gerenciada por um *event loop*, e as tarefas (corrotinas) cedem voluntariamente o controle (cooperativamente) através da palavra-chave `await`.

- **`asyncio.queue`:** Esta fila é "segura" por natureza dentro do seu event loop. Ela gerencia internamente a lógica de "buffer cheio" e "buffer vazio".
- **`await queue.put()`:** Se a fila estiver cheia, esta chamada *automaticamente* pausa a corrotina do produtor e cede o controle ao event loop, que executa outra tarefa.

- **await queue.get():** Se a fila estiver vazia, esta chamada *automaticamente* pausa o consumidor e cede o controle.

2. Demonstração de corretude

- **Na versão preemptiva (threads),** a corretude é garantida pela combinação das primitivas. O Lock previne que dois threads modifiquem o buffer ao mesmo tempo (condição de corrida), e os Semáforos empty e full previnem que produtores adicionem a um buffer cheio ou que consumidores removam de um buffer vazio.
- **Na versão cooperativa (coroutines),** a corretude é garantida pelo design. Como tudo roda em um único thread, é *impossível* ter uma condição de corrida (duas operações ocorrendo no mesmo instante). O event loop só troca de tarefa em pontos definidos (await), eliminando a necessidade de Locks para proteger a fila.

3. Diferenças de desempenho

3.1. Metodologia de Teste

Decidi utilizar 6 cenários com valores definidos no código, para a quantidade de consumidores, produtores, tamanho do buffer e itens com valores estáticos ao invés de pedir uma entrada para defini-los, pois desta maneira posso simplificar a execução do programa, necessitando apenas rodar o arquivo "run.experiments()", ainda sim os valores podem ser alterados modificando os valores no código executar_testes.py.

3.1 Análise dos Resultados

Os experimentos foram executados 5 vezes para 6 cenários diferentes, simulando tarefas de I/O (entrada/saída) usando um número aleatório em um intervalo de 0.,001 até 0,02 para simular uma tarefa real de I/O, através de breves "esperas" (time.sleep e asyncio.sleep). Os dados completos estão em resultados_benchmark.csv.

Abaixo está uma tabela resumida dos **tempos médios de execução (em segundos)**:

Cenário	Tarefas (P+C)	Itens (Total)	Corrotinas (Média)	Threads (Média)	Vencedor
1	3P + 3C	60	0.2356s	0.2306s	Threads
2	10P + 10C	500	0.6025s	0.5723s	Threads
3	5P + 5C	1000	2.3058s	2.1904s	Threads
4	50P + 50C	500	0.1405s	0.1771s	Corrotinas

5	20P + 5C	1000	2.2841s	2.1819s	Threads
6	5P + 20C	250	0.3459s	0.3341s	Threads

Os resultados são extremamente reveladores.

Na maioria dos cenários (1, 2, 3, 5, 6), a implementação com threads foi ligeiramente mais rápida do que a implementação com corrotinas. Isso ocorre porque, para um número baixo de threads (de 6 a 25), o Sistema Operacional é muito eficiente em gerenciar o bloqueio de I/O (`time.sleep`), e o pequeno custo de gerenciamento do *event loop* do `asyncio` o torna marginalmente mais lento.

No entanto, o **Cenário 4 ("Muitas Tarefas")** prova o ponto central da concorrência moderna. Neste cenário, criamos 100 tarefas simultâneas (50 Produtores + 50 Consumidores).

- A versão com **Corrotinas (asyncio)** foi a **vencedora clara** (aprox. 25% mais rápida). Criar 100 "tarefas" no `asyncio` é extremamente leve e a troca de contexto entre elas é quase instantânea.
- A versão com **Threads** foi mais lenta porque criar 100 threads reais do SO é uma operação "cara". O Sistema Operacional gasta um tempo significativo apenas *gerenciando* esses threads e realizando a "troca de contexto" (parar um thread, salvar seu estado, carregar o estado de outro), que é um processo muito mais pesado do que a troca de contexto de um `event loop`.

4. Situações em que cada abordagem é mais apropriada

Baseado nos dados, podemos concluir:

- **Threads (Preemptivo):** É uma abordagem adequada quando o número de tarefas de I/O é **pequeno, conhecido e fixo** (ex: 5-20 tarefas). A sobrecarga (overhead) do SO é baixa e a lógica pode ser mais direta para quem não está acostumado com `async/await`.
- **Corrotinas (Cooperativo):** É a abordagem **superior** para aplicações que precisam de **alta escalabilidade e gerenciam um número massivo (centenas, milhares ou dezenas de milhares)** de conexões ou tarefas de I/O simultâneas. É a escolha padrão para servidores web modernos, APIs e aplicações de rede, pois seu custo de gerenciamento de tarefas é quase zero.